

Deterministic Audio Latency Analysis on Linux-Based SoCs

Experimental Characterization with Renode, PetaLinux, and PREEMPT_RT

Yasin Arslan* and Suleyman Sertac Ust†

*Computer Engineering Department, Hacettepe University, Ankara, Turkey

†Graduate School of Science and Engineering, Hacettepe University, Ankara, Turkey

E-mails: yasinarslan@hacettepe.edu.tr, ssertacust26@hacettepe.edu.tr

Abstract—Safety-critical communication systems increasingly rely on Linux-based heterogeneous architectures to combine software flexibility with high computational capacity. However, the general-purpose Linux scheduling and I/O stack can introduce latency spikes and jitter that are problematic for real-time audio pipelines. This report extends the original proposal into a final implementation and evaluation of a reproducible simulation-based framework for audio timing analysis on a Zynq UltraScale+ MPSoC-like platform. The implemented system uses Renode for simulation, PetaLinux for the guest Linux environment, ALSA for the software audio path, and the Linux `snd-dummy` driver as a practical replacement for a fully modeled physical audio peripheral. Experiments compare a standard Linux kernel and a PREEMPT_RT patched kernel across three ALSA timing configurations and four synthetic load profiles. Results show that average buffered audio delay is largely governed by the selected ALSA period and buffer configuration, while tail latency, worst-case service duration, and XRUN frequency reveal meaningful differences between standard Linux and PREEMPT_RT under stress. A planned fine-grained explanation of exactly which pipeline layer caused each delay spike could not be completed within the project schedule. The main outcome is a working experimental framework, evidence-based comparative analysis, and a clear account of what remains before full latency-source attribution and hardware-level validation.

Index Terms—Real-Time Systems, Embedded Linux, Audio Pipelines, PREEMPT_RT, ALSA, PetaLinux, Renode Simulation, Zynq MPSoC, XRUN Analysis.

I. INTRODUCTION

Modern embedded communication systems are moving from dedicated microcontrollers and proprietary real-time operating systems toward heterogeneous system-on-chip platforms running embedded Linux. This transition provides a mature software ecosystem, strong networking support, and high processing capability. At the same time, it introduces a central real-time design problem: standard Linux is optimized mainly for throughput and fairness, not strict timing guarantees. In audio communication, late frames can cause audible glitches, underruns, deadline misses, or loss of temporal synchronization.

This project investigates deterministic audio latency on Linux-based SoCs. The original proposal identified a research gap in reproducible, simulation-based end-to-end audio latency analysis. The final implementation focuses on a practical subset of that proposal: building a Renode and PetaLinux-based experiment environment, running ALSA timing workloads through `snd-dummy`, applying controlled system stress, and

comparing a standard Linux kernel with a PREEMPT_RT kernel.

The final project makes four main contributions:

- A working simulation-oriented experimental setup using Renode, PetaLinux, ALSA, and `snd-dummy`.
- A parameterized test plan covering three ALSA timing configurations and four load profiles.
- A comparative analysis of standard Linux and PREEMPT_RT using mean service time, P99 latency, maximum latency, jitter/spread, and XRUN frequency.
- A clear discussion of achieved components, Renode integration issues, and the remaining gap of fine-grained latency-source attribution.

The central argument of the project is that deterministic audio should not be judged only by average latency. Tail behavior, maximum latency, jitter, and XRUN events provide a more meaningful view of whether a Linux-based audio pipeline can consistently meet real-time requirements. The original proposal also aimed to explain where the delay accumulates inside the audio path; in the final work, this goal is reported honestly as unfinished because it could not be completed within the available project time.

II. RELATED WORK

The problem addressed in this project relates to three areas: real-time Linux, low-latency Linux audio, and simulation-based embedded system validation.

PREEMPT_RT modifies the Linux kernel to improve pre-emptibility, convert many interrupt handlers into kernel threads, and reduce priority inversion effects. Surveys of real-time Linux show that PREEMPT_RT can significantly reduce worst-case scheduling latency, but they also emphasize that it does not automatically provide hard real-time guarantees for every application stack [?]. This distinction is important for audio systems because application-level behavior depends not only on scheduler latency but also on buffering, driver behavior, userspace wakeups, and system load.

Linux audio frameworks such as ALSA expose period size, buffer size, and start threshold parameters that directly affect latency and robustness. Prior work on low-latency audio systems emphasizes the trade-off between smaller buffers and increased risk of underruns [?], [?], [?]. Smaller periods reduce

nominal latency, but they also make the audio pipeline more sensitive to scheduling delay and CPU contention.

Simulation frameworks such as Renode provide repeatable execution environments for embedded systems. Renode is not a cycle-accurate physical model of all SoC components, but it is useful for controlled software-oriented experiments because it can boot embedded Linux images, model key platform components, and provide deterministic simulation behavior [?]. Timing-analysis literature also shows that simulation and analytical models can help isolate latency sources before hardware validation [?], [?].

This project combines these directions by using simulation to compare Linux kernel configurations under controlled audio workloads. Compared with the original proposal, the final implementation uses Linux `snd-dummy` rather than a fully modeled audio codec or I2S path. This change makes the experiment feasible while keeping the ALSA and kernel software timing path observable.

III. PROBLEM DEFINITION

A. Target Problem

The target problem is the characterization of timing predictability in a Linux-based real-time audio pipeline. A real-time audio system must not only process audio quickly on average; it must bound its latency sufficiently so that buffer servicing and playback deadlines are not missed. Standard Linux can exhibit latency spikes when background work, memory pressure, or scheduling contention interferes with an audio task.

B. Objectives

The final implementation follows these objectives:

- 1) Boot and configure a PetaLinux-based Linux system in a Renode-based ZynqMP environment.
- 2) Enable an ALSA-compatible audio path using `snd-dummy`.
- 3) Execute the same timing tests under standard Linux and PREEMPT_RT kernels.
- 4) Apply repeatable load profiles ranging from idle to extreme stress.
- 5) Analyze service duration, buffered delay, P99 latency, maximum latency, jitter/spread, and XRUN events.
- 6) Report the current status of the planned latency-source explanation.

C. Determinism Criteria

Let L_i denote the measured service duration of the i -th audio write operation in the collected CSV logs, computed from the `write_duration_ns` field. Let D_i denote the ALSA-estimated buffered delay converted from `delay_frames` to milliseconds. We report mean, standard deviation, P99, and maximum values for L_i , while D_i is used to interpret the latency implied by the selected ALSA buffer configuration. XRUN frequency is used as a reliability metric because buffer underrun or overrun events indicate that the audio pipeline failed to meet timing requirements.

The original proposal used a strict interactive voice target of $L_{\max} = 10$ ms and a jitter threshold of $J_{\text{th}} = 1$ ms. In the final implementation, these thresholds are interpreted cautiously because `snd-dummy` does not represent a complete physical audio path. Therefore, the final evaluation focuses primarily on comparative determinism: whether PREEMPT_RT reduces tail latency and XRUN frequency relative to standard Linux under identical workloads.

D. Latency-Source Attribution Scope

A planned part of the original methodology was to explain not only *how much* latency was observed, but also *where* that latency came from. The intended model separated the pipeline into application processing, ALSA buffering, kernel scheduling, context switching, driver/interrupt handling, and hardware or peripheral delay.

In the final implementation, this layer-by-layer latency-source attribution could not be fully completed within the available project time. Therefore, the report treats latency-source attribution as an unfinished part of the original plan rather than presenting a detailed root-cause explanation. The empirical results are still used to compare standard Linux and PREEMPT_RT under identical workloads, but they are not claimed to fully explain the origin of every individual latency spike.

IV. SYSTEM ARCHITECTURE AND DESIGN

A. Implemented Platform

The implemented platform is shown conceptually in Fig. 1. The simulation stack starts from Renode, which is used to emulate the target ZynqMP/ZCU102-style platform. PetaLinux provides the embedded Linux environment, and ALSA provides the software audio framework. Instead of a fully modeled physical audio codec and I2S/DMA chain, the Linux `snd-dummy` driver creates a dummy ALSA sound card that can be used for software-layer audio timing experiments.

The use of `snd-dummy` is an important design decision. The original plan included a more realistic virtual or physical audio device path. However, a complete ZCU102 audio chain would require practical modeling of codec behavior, clocks, DMA interactions, and low-level audio peripherals inside Renode. That was beyond the feasible scope of the course project. `snd-dummy` removes the need for the physical audio peripheral while preserving the ALSA and kernel scheduling path needed for timing comparison.

B. Renode Platform Bring-Up and Integration Challenges

Renode was not only used as a passive execution environment; a significant part of the implementation was the adaptation of the PetaLinux image to a practical simulated ZynqMP platform. The device tree, platform description, and root filesystem had to describe a system that Renode could boot reliably while still preserving the Linux, ALSA, and scheduling layers needed for the experiment.

One important class of issues was related to timer and interrupt configuration. The initial PetaLinux device tree

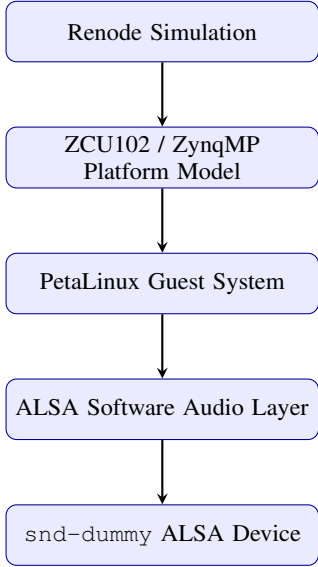


Fig. 1. Implemented software-oriented audio timing platform.

contained timer resources that were not fully aligned with the available Renode model. Since Linux timekeeping and scheduler wakeups depend on stable timer interrupts, unsupported or mismatched timer nodes could lead to unstable boot behavior or unreliable runtime timing. The final setup avoided unnecessary timer sources and kept the timer configuration consistent with the Renode-supported ZynqMP model.

A second class of issues was related to GEM Ethernet controllers. Some GEM instances were not needed for the audio latency measurements but could still affect the boot path through PHY, MDIO, clock, reset, or network-service dependencies. Instead of spending project time on a complete Ethernet path, the final configuration removed networking from the critical experiment path and disabled or ignored non-essential GEM-related components. This also reduced boot-time complexity and avoided unrelated service delays.

TABLE I
RENODE AND PETA LINUX BRING-UP ISSUES

Issue	Resolution / Design Decision
Timer mismatch	Aligned the device-tree timer setup with the timers supported by the Renode ZynqMP model; avoided unsupported timer nodes.
GEM bring-up	Removed Ethernet from the critical path; disabled or ignored non-essential GEM and network-service dependencies.
Peripheral coverage	Disabled hardware blocks not required for Linux boot, ALSA, <code>snd-dummy</code> , and timing tests.
Rootfs overhead	Reduced the root filesystem and kept only the services and modules required by the experiment.

The `snd-dummy` module also had to be explicitly loaded and verified under `/proc/asound/cards`. These bring-up decisions made the simulated environment narrower than a full ZCU102 model, but they made it more reproducible and better focused on the software timing path that the project evaluates.

V. METHODOLOGY

A. Experimental Design

Each experiment is defined by three dimensions: kernel type, ALSA timing configuration, and load profile. The two kernels are standard Linux and PREEMPT_RT. The timing configurations are listed in Table II. These configurations intentionally explore the trade-off between conservative buffering and aggressive low-latency execution.

TABLE II
ALSA TIMING CONFIGURATIONS USED IN THE FINAL EXPERIMENTS

Profile	Period	Buffer	Thr.	Delay	Purpose
Safe	128	512	128	~2.66 ms	Conservative reference setting.
Low	64	256	64	~1.33 ms	Aggressive low-latency setting.
Ultra-Low	32	128	32	~0.66 ms	Tight buffering to expose breaking points.

The load profiles are shown in Table III. For every timing configuration, the same load profiles were executed for both kernels. Therefore, a result file such as `safe_heavy.csv` means the Safe timing configuration tested under the Heavy load profile. The full matrix contains 12 CSV files per kernel.

TABLE III
SYSTEM LOAD PROFILES

Load	Meaning	Purpose
Idle	No synthetic stress.	Baseline behavior.
Light	Moderate CPU activity.	Normal multitasking case.
Heavy	CPU and memory pressure.	Observe latency and jitter growth.
Extreme	Near-saturation stress.	Expose worst-case limits.

B. Measurement Metrics

The collected CSV logs contain 1000 iterations per experiment. The main fields used in this report are `write_duration_ns`, `delay_frames`, and `xrun_count`. `write_duration_ns` is converted to milliseconds and used as the main service-duration metric. `delay_frames` is converted to milliseconds using a 48 kHz sample rate and used to interpret the buffered delay imposed by ALSA configuration. `xrun_count` is differenced over a run to count XRUN events.

The primary metrics are mean service duration, maximum service duration, 99th percentile service duration, standard deviation as a jitter/spread proxy, estimated buffered delay, and XRUN increment. This metric set follows the main project principle: deterministic audio must be evaluated by tail behavior and failure events, not only by the mean.

C. What Was Not Measured

The original proposal included a more detailed explanation of where latency accumulates inside the audio path. This part could not be completed before the final report deadline. For

that reason, the final CSV files are used for comparative performance evaluation, but not for a complete causal breakdown of every latency spike.

As a result, the report makes direct claims about measured behavior, such as changes in P99 latency, maximum service duration, and XRUN count. It does not claim to prove the exact internal source of each observed delay event. This remaining gap is listed explicitly as future work.

D. Implementation Status

Table IV summarizes the final status of the original proposal components.

VI. EXPERIMENTAL SETUP

A. Tools and Environment

The final environment used Renode as the simulator, PetaLinux as the embedded Linux build and boot environment, ALSA as the audio framework, and Linux `snd-dummy` as the ALSA-compatible dummy sound card. The same audio timing workload was run for standard Linux and PREEMPT_RT. The two result archives contain matching CSV names for the two kernels, enabling direct comparison.

The selected ALSA parameters are period size, buffer size, and start threshold. Safe uses larger buffers, Low uses smaller buffers, and Ultra-Low uses the tightest buffers. This setup intentionally creates a design trade-off: larger buffers provide robustness but increase buffered delay, while smaller buffers reduce nominal delay but can increase XRUN risk under stress.

B. Data Processing

For each CSV file, rows after the metadata header were parsed as measurement samples. The following transformations were applied:

$$L_i [\text{ms}] = \frac{\text{write_duration_ns}_i}{10^6} \quad (1)$$

$$D_i [\text{ms}] = \frac{\text{delay_frames}_i}{48000} \times 1000 \quad (2)$$

XRUN increment was computed as $\max(\text{xrun_count}) - \min(\text{xrun_count})$ for each run. P99 values were computed from the sorted service-duration samples.

VII. SYSTEM-LEVEL CONSIDERATIONS

A. Real-Time Behavior

The experiment directly reflects a core embedded real-time design trade-off. PREEMPT_RT can improve scheduler responsiveness and reduce tail latency, but real-time behavior also depends on buffer size, userspace wakeups, IRQ handling, and overall system contention. Therefore, the PREEMPT_RT kernel is evaluated as one part of the timing system rather than as a complete deterministic guarantee.

B. Reliability

XRUN events are the main reliability-oriented metric in the final results. A system can have acceptable average latency while still failing if the audio buffer is not serviced in time. In this project, XRUN analysis complements mean, maximum, and P99 latency measurements by identifying actual timing failures in the audio pipeline.

C. Resource Constraints

The three timing profiles demonstrate the resource trade-off between latency and robustness. Safe buffering produces larger estimated buffered delay but fewer failures. Ultra-Low buffering reduces the nominal delay target but becomes much more sensitive to scheduling and load, especially in the standard kernel. This is a typical embedded design decision: the most aggressive configuration is not necessarily the most reliable one.

D. Simulation Limits

Renode provides reproducibility and a controllable software environment, but it does not fully model physical timing fidelity, thermal effects, electrical behavior, all clocking details, or a complete audio codec/DMA chain. Similarly, `snd-dummy` is useful for ALSA and kernel-level experiments but does not represent all real hardware delays. For this reason, the final conclusions are limited to the simulated software audio path.

E. Latency Attribution Limitation

The results should be read as a comparison of observable timing behavior, not as a complete diagnosis of every latency source. The original plan included a more detailed explanation of where latency accumulates inside the audio pipeline, but this part could not be completed within the available project time. Therefore, latency-source attribution remains a future-work item.

VIII. RESULTS AND EVALUATION

A. Performance Results

The final experiments produced 24 CSV files: 12 for the standard kernel and 12 for the PREEMPT_RT kernel. Each kernel was tested across Safe, Low, and Ultra-Low timing configurations and Idle, Light, Heavy, and Extreme load profiles.

Table V summarizes the most important stress case: Extreme load. Under this condition, standard Linux exhibits larger tail and maximum service duration than PREEMPT_RT in all three timing configurations. The difference is especially large for the Ultra-Low configuration, where the standard kernel reaches a P99 service duration of 5.013 ms and maximum of 9.100 ms, while PREEMPT_RT reduces these to 1.400 ms and 2.000 ms, respectively.

Fig. 2 visualizes the same extreme-load tail and worst-case behavior. The figure shows that PREEMPT_RT does not substantially change the mean service time for a given timing profile, but it strongly reduces the worst-case and P99 service durations when the system is stressed.

TABLE IV
PROPOSAL VERSUS FINAL IMPLEMENTATION STATUS

Proposal Item	Status	Evidence / Comment
Renode simulation framework	✓	Platform booted and used in the experiment workflow.
PetaLinux boot and configuration	✓	ZynqMP image configured with required modules.
ALSA audio pipeline execution	✓	Achieved with the Linux <code>snd-dummy</code> ALSA driver.
Standard vs PREEMPT_RT comparison	✓	Matching CSV data collected for both kernels.
Stress-profile testing	✓	Four load profiles executed for each timing profile.
Full physical audio path	–	Simplified; the codec/I2S/DMA path was not modeled in Renode.
Predictive latency budget validation	Partial	Concept retained, but complete quantitative validation remains future work.
Latency-source attribution	Not completed	Planned root-cause explanation could not be finished within the project time.

TABLE V
SERVICE-DURATION RESULTS UNDER EXTREME LOAD

Timing Profile	Kernel	Mean (ms)	P99 (ms)	Max (ms)	Std. Dev. (ms)	XRUN Increment
Safe	Standard	2.556	6.300	7.600	0.888	1
Safe	PREEMPT_RT	2.557	5.600	5.600	0.909	0
Low	Standard	1.233	5.000	9.000	0.828	4
Low	PREEMPT_RT	1.221	3.503	6.200	0.663	0
Ultra-Low	Standard	0.609	5.013	9.100	0.834	19
Ultra-Low	PREEMPT_RT	0.613	1.400	2.000	0.393	0

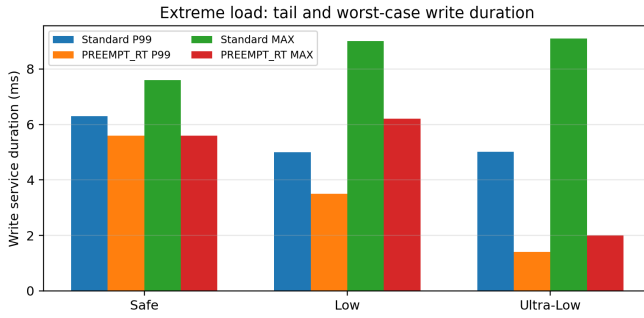


Fig. 2. P99 and maximum service duration under the Extreme load profile.

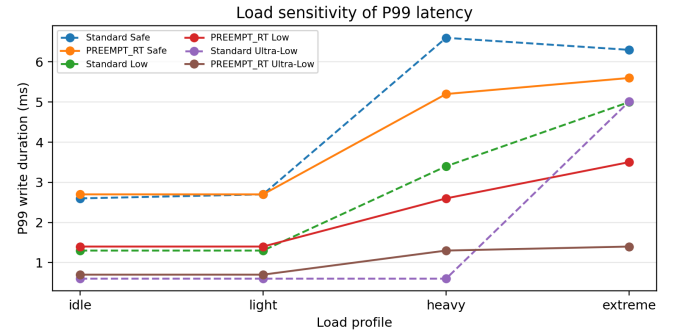


Fig. 3. Load sensitivity of P99 service duration across timing configurations.

The load-sensitivity trend is shown in Fig. 3. P99 values remain close to the configured timing behavior in idle and light conditions, but they grow under heavy and extreme stress. The standard kernel is more sensitive to this growth, while PREEMPT_RT keeps the tail more bounded, particularly in the Low and Ultra-Low configurations.

XRUN behavior is summarized in Fig. 4. Standard Linux accumulates XRUN events mostly under the more aggressive Low and Ultra-Low configurations. Across all loads, standard Linux records 4 XRUN increments in Low and 21 in Ultra-Low, while PREEMPT_RT records 0 in Low and 2 in Ultra-Low. Safe remains the most robust configuration, but also has the largest buffered delay. These results identify when the pipeline failed, but not the exact internal layer that caused each failure.

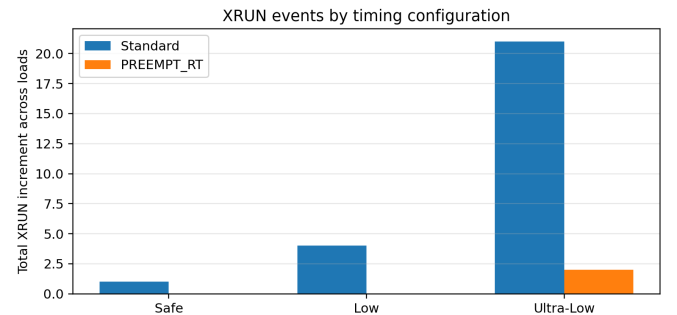


Fig. 4. Total XRUN increment across all load profiles for each timing configuration.

B. Comparative Analysis

The most meaningful baseline is the standard Linux kernel under the same timing configuration and load profile. Compared with this baseline, PREEMPT_RT improves tail behavior under extreme contention. For the Low configuration, P99 service duration is reduced from 5.000 ms to 3.503 ms, a reduction of approximately 29.9%, and maximum service duration is reduced from 9.000 ms to 6.200 ms. For the Ultra-Low configuration, the P99 reduction is approximately 72.1%, and the maximum reduction is approximately 78.0%. These results support the expectation that PREEMPT_RT improves temporal predictability under contention.

However, the results also show that buffering configuration remains a dominant design factor. The estimated buffered delay from `delay_frames` is governed primarily by period and buffer sizes: Safe produces a mean buffered delay of approximately 10.2–10.5 ms, Low approximately 4.9–5.1 ms, and Ultra-Low approximately 2.2–2.5 ms. This behavior is expected because `snd-dummy` uses ALSA buffer accounting rather than a physical playback chain. Therefore, PREEMPT_RT mainly improves the tail of the service-duration distribution and the XRUN behavior; it does not rewrite the latency implied by the selected buffer configuration.

The planned predictive latency budget was also intended to support a detailed explanation of latency sources. This part was not completed within the project time. Therefore, the final comparison focuses on the measured differences between kernels and timing profiles, while the detailed source-level explanation of delay is left as future work.

The comparison between expected and obtained behavior is mostly consistent with the proposal. The expected trend was that PREEMPT_RT would reduce jitter and improve worst-case bounds, while increased system load would increase variability. The final data supports this trend. The main difference from the proposal is that the complete physical audio path was not implemented, so the analysis is restricted to the ALSA/software timing path.

C. Discussion

Several parts of the project worked as expected. The Renode and PetaLinux environment could be used to run repeatable Linux audio experiments. The `snd-dummy` driver allowed the ALSA layer to be exercised without requiring a modeled physical audio device. The experimental matrix of two kernels, three timing profiles, and four load profiles produced comparable CSV outputs for analysis.

The main technical finding is that PREEMPT_RT improves timing predictability primarily at the tail. Average service duration remains close between kernels because it is strongly influenced by the selected ALSA parameters and the dummy device behavior. The tail metrics tell a different story: under heavy and extreme loads, standard Linux exhibits larger spikes, while PREEMPT_RT reduces P99 and maximum service durations in the most stressful cases. XRUN results reinforce this interpretation because standard Linux shows more timing failures in aggressive configurations.

The main trade-off is between latency and robustness. Safe buffering is stable but has a larger buffered delay. Ultra-Low buffering is attractive for low latency, but it becomes fragile under stress, especially with the standard kernel. PREEMPT_RT improves the viability of more aggressive settings, but it does not eliminate the need for careful buffer sizing and workload control.

The largest limitation is simulation fidelity. Renode gives reproducibility, but the final system does not include a physical codec, a real I2S controller, a real DMA engine, or physical clocking effects. The `snd-dummy` device is therefore a software-layer approximation. This limitation does not invalidate the comparison between standard Linux and PREEMPT_RT under the same conditions, but it prevents direct claims about final real-hardware audio latency on a physical ZCU102 board.

Another limitation is that the predictive latency budget and latency-source attribution were not fully completed. The original plan was to explain where latency accumulates inside the full audio pipeline, but this part could not be finished within the available project time. The project still provides a useful intermediate result: a reproducible measurement pipeline and a quantitative comparison that can be extended later for detailed root-cause analysis.

From an embedded systems design perspective, the results emphasize real-time behavior, reliability, and resource constraints. PREEMPT_RT helps with real-time scheduling predictability, XRUNs reveal reliability failures, and the timing profiles expose the buffer-size resource trade-off. Energy efficiency and security were considered conceptually in the proposal, but they were not measured in the final implementation.

D. Reproducibility

The experiments can be reproduced by preparing two matching Linux images: one standard PetaLinux kernel and one PREEMPT_RT patched kernel. Both images should include ALSA userspace tools and the Linux `snd-dummy` module. The same Renode script should boot the same ZynqMP/ZCU102-style platform model for both kernels, and unnecessary peripherals should be disabled if they prevent boot or add unrelated simulation overhead.

The reproduction workflow is:

- 1) Start from the Renode platform script and device tree used in this report, including the simplified timer and non-essential GEM/network configuration.
- 2) Boot the selected kernel in Renode with the PetaLinux root filesystem.
- 3) Load `snd-dummy` and verify that a dummy sound card appears under `/proc/asound/cards`.
- 4) Configure ALSA with the period, buffer, and start-threshold values listed in Table II.
- 5) Run the timing test for 1000 iterations under each load profile and export the CSV log.
- 6) Repeat the same 12 experiments for the standard kernel and the PREEMPT_RT kernel.

- 7) Parse write duration, delay frames, and XRUN count; then compute mean, P99, maximum, standard deviation, buffered delay, and XRUN increment.
- 8) Complete the planned latency-source attribution step in a future version of the project.

The dataset names combine the timing profile and load profile. For example, Safe under Heavy load is `safe_heavy.csv`, Low under Idle load is `low_idle.csv`, and Ultra-Low under Extreme load is `ultra_extreme.csv`. The standard-kernel and PREEMPT_RT archives use the same file names, allowing direct comparison. The code was not published in a public repository at the time of writing; therefore, this report lists the workflow and parameters explicitly.

IX. CONCLUSION AND FUTURE WORK

This project addressed the problem of deterministic audio timing on Linux-based embedded SoCs. The original proposal targeted a dual methodology combining empirical measurement and predictive latency budgeting. The final implementation successfully realized the empirical part of the methodology in a Renode and PetaLinux-based environment using ALSA and `snd-dummy`. The project also completed a controlled comparison between standard Linux and PREEMPT_RT across three timing configurations and four load profiles.

The main finding is that PREEMPT_RT improves timing predictability under stress, especially in tail latency and XRUN behavior. Under Extreme load, PREEMPT_RT reduces P99 and maximum service duration relative to standard Linux in all timing profiles, with the strongest improvement in the Ultra-Low configuration. At the same time, the results show that average buffered delay is dominated by ALSA buffer configuration, so kernel real-time patching alone is not sufficient to guarantee deterministic audio.

The main completed components are the Renode/PetaLinux setup, ALSA `snd-dummy` execution, standard versus PREEMPT_RT comparison, stress-profile execution, CSV data collection, and quantitative results analysis. The main components that could not be fully completed are the realistic physical audio device model, the complete validation of the predictive latency budget, and the original goal of explaining exactly where latency accumulates inside the full audio pipeline. The latency-source explanation was not finished within the project time, so the final report separates measured comparative behavior from this unfinished goal.

Future work should extend the current framework in four directions. First, the `snd-dummy` path should be replaced or complemented by a more realistic audio device model or real ZCU102 hardware validation. Second, the predictive latency budget should be finalized and compared directly with empirical distributions. Third, the latency-source explanation that was planned in the proposal should be completed. Fourth, the experiment scripts should be packaged into a public repository with automated execution and plotting so that all results can be reproduced more easily. Beyond the course project, additional

work could investigate CPU affinity, real-time priorities, multi-channel audio scalability, and energy-aware scheduling policies.

ACKNOWLEDGMENTS

The authors would like to thank the CMP720 Embedded System Design course staff for their guidance and evaluation framework. We also acknowledge the open-source communities behind Linux, ALSA, stress-ng, and Renode, whose tools made this simulation-based project possible.

ibliographystyleIEEEtran ibliographyreferences